

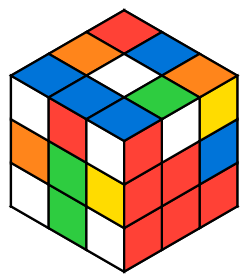
magic-cubes

v0.0.1 MIT

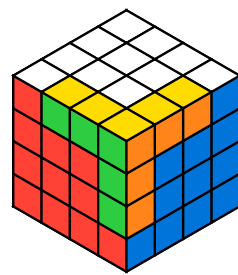
Represent Rubik's cubes of arbitrary size and apply algorithms to them

Rodrigo Alcántara

 @RodAlc24



U' L' U' F' R2 B' R
F U B2 U B' L U' F
U R F'



F U2 L F L' B L U
B' R' L' U R' D' F'
B R2

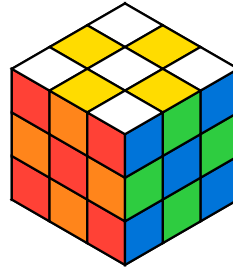
Table of Contents

Examples	2
Guide	4
II.1 Creating cubes	4
II.2 Applying algorithms	5
II.3 Rendering cubes	6
Cube notation	7
III.1 1-layer moves	7
III.2 Central moves	8
III.3 Wide moves	8
III.4 Cube rotations	9
III.5 Modifiers	9
Reference	10
IV.1 Custom types	10
IV.1.1 <code>cube</code>	10
IV.1.2 <code>cube-colors</code>	10
IV.1.3 <code>cube-state</code>	10
IV.2 Functions	11
Index	15

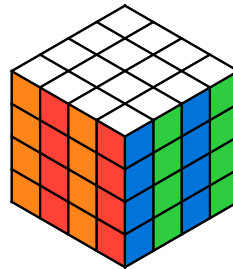
Part I

Examples

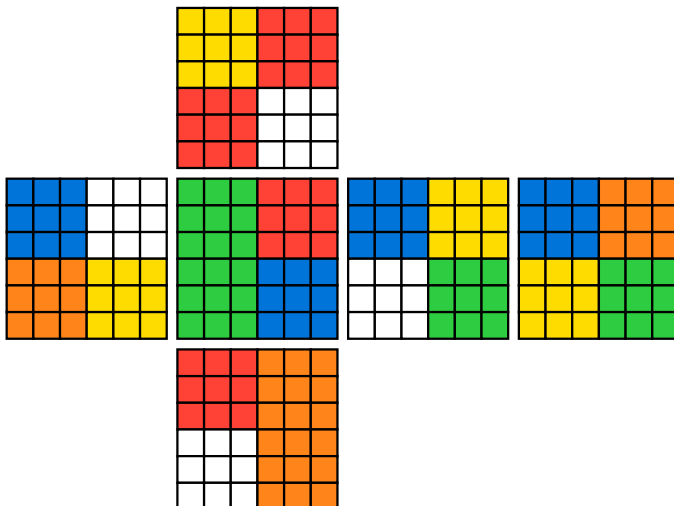
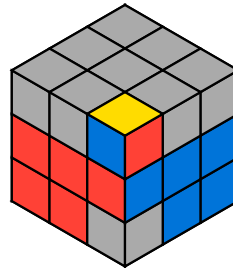
```
#draw_cube(
  apply(
    cube(),
    "F2 B2 R2 L2 U2 D2"
  )
)
```



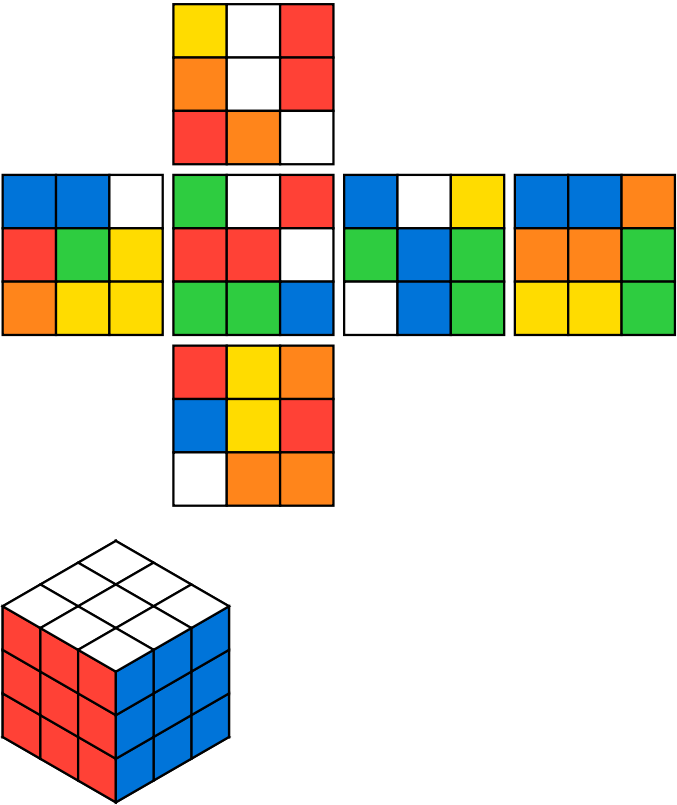
```
#draw_cube(
  apply(
    cube(size: 4),
    "R2 3R2 F2 3F2 R2 3R2"
  )
)
```



```
#draw_cube(
  apply(
    inverted: true,
    f2l-cube,
    "R2 U R2 U R2 U2 R2"
  )
)
```



I Examples



Part II

Guide

II.1 Creating cubes

A cube is represented as a dictionary, and can be easily created with the `#cube` function. It should not be created or modified manually.

The size of the cube can be specified with the `<size>` argument (default is 3). The face colors can be changed with the `<colors>` argument, specifying the faces to change. The default colors are:

```
(  
  f: red,  
  r: blue,  
  u: white,  
  b: orange,  
  l: green,  
  d: yellow  
)
```

This colors are the Typst [predefined colors](#)¹.

It is also possible to specify a face manually, in that case the corresponding value in `<colors>` is ignored. When specifying the colors manually, all the pieces in the face must be determined.

The order of the pieces in a face is the following:

			0	1	2			
			3	4	5			
			6	7	8			
0	1	2	0	1	2	0	1	2
3	4	5	3	4	5	3	4	5
6	7	8	6	7	8	6	7	8
			0	1	2			
			3	4	5			
			6	7	8			

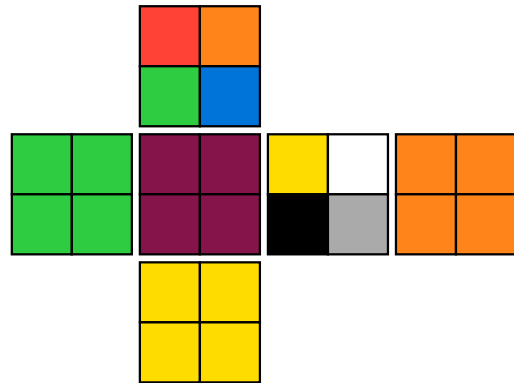
Each face is represented with the default color previously explained, except the up face that is in black.

Note that this way of creating cubes may result in an impossible cube state (you can even use more than six colors!). This can be useful, for example, to add a gray color to the non relevant pieces.

¹<https://typst.app/docs/reference/visualize/color/#predefined-colors>

This order is also used in other size cubes, for example, if we want to create a 2x2x2 cube with a custom color in front face and custom up and right faces we can do:

```
#draw_flat(
  length: 15pt,
  cube(
    size: 2,
    colors: (f: maroon),
    state: (
      u: (
        red,
        orange,
        green,
        blue,
      ),
      r: (
        yellow,
        white,
        black,
        gray,
      ),
    ),
  )
)
```



There are also some useful predefined cubes: f2l-cube and oll-cube.

```
#draw_cube(f2l-cube, length: 10pt)
```



```
#draw_cube(oll-cube, length: 10pt)
```

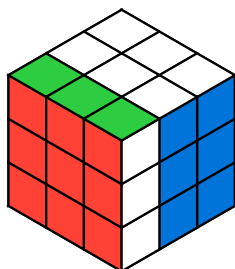


II.2 Applying algorithms

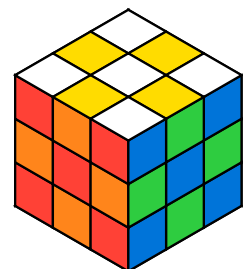
One of the main features of [magic-cubes](#) is the possibility to apply algorithms to the cubes. Algorithms are written following the standard notation that is fully documented on [Section III](#).

The function `#apply` receives a cube and an algorithm as a `str`. It is also possible, through the `(inverted)` argument to apply the inverse algorithm. This results in a cube that, after applying the specified algorithm, will result in the original cube.

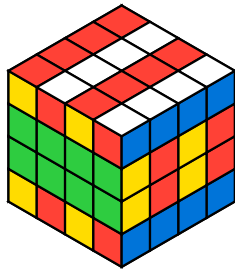
```
#draw_cube(
  apply(
    cube(),
    "F"
  )
)
```



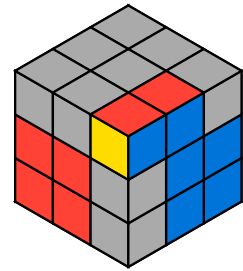
```
#draw_cube(
  apply(
    cube(),
    "M2 E2 S2"
  )
)
```



```
#draw_cube(  
  apply(  
    cube(size: 4),  
    "2R L' 2-3u'"  
  )  
)
```



```
#draw_cube(  
  apply(  
    f2l-cube,  
    inverted: true,  
    "U R U' R'"  
  )  
)
```



There are also two alternative functions that lets you modify the cube: `#rotate_layer` and `#rotate_cube`.

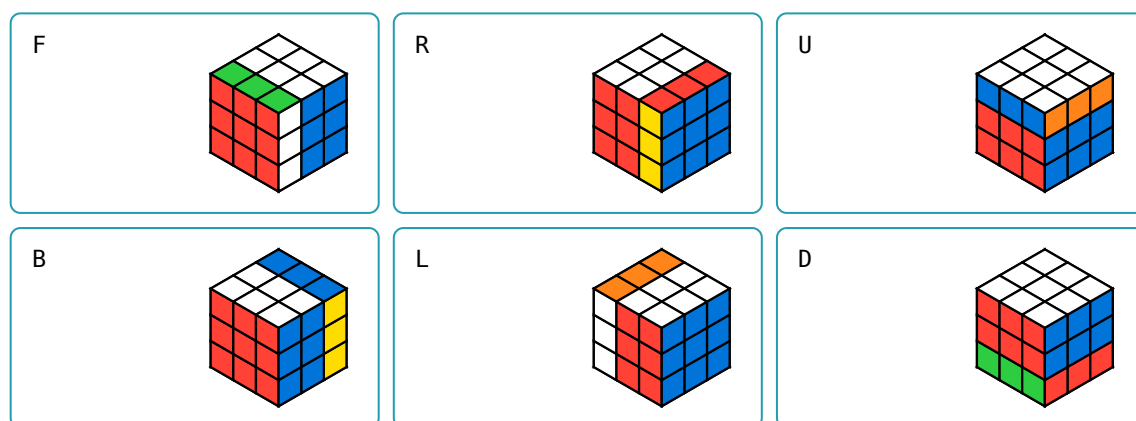
II.3 Rendering cubes

Part III

Cube notation

III.1 1-layer moves

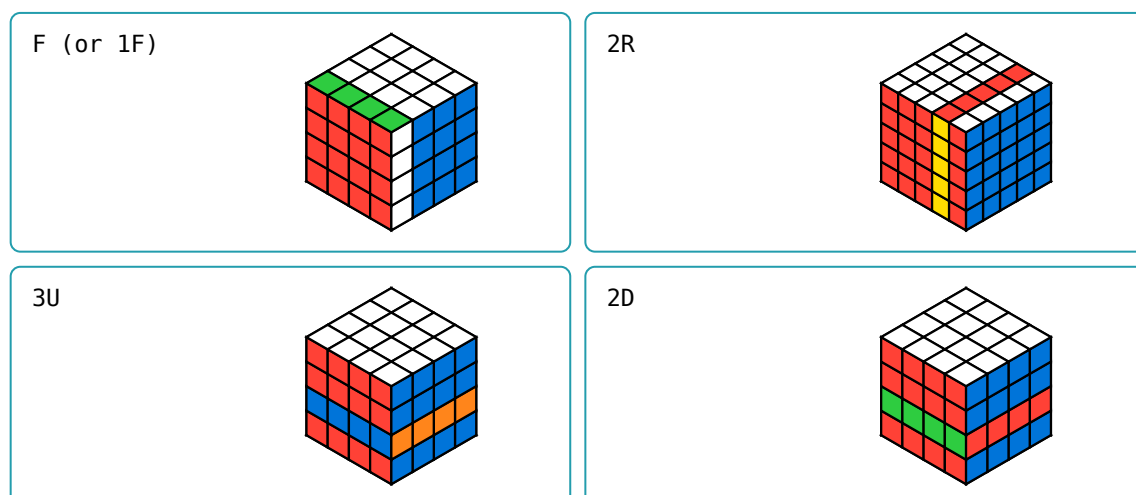
The basic moves are represented with one uppercase letter. There are six moves, one for each face of the cube: **F** (front), **R** (right), **U** (up), **B** (back), **L** (left) and **D** (down). Each represents a single clockwise rotation. Double and counterclockwise rotations are explained in [Section III.5](#).



It is also possible to move an inner layer. To do so, precede the letter with the depth of the layer. The outermost layer has a depth of 1 and layer numbering always starts from the referenced face.

It is not necessary to specify the depth for the moves of the outermost layer. For example, 1U is equivalent to U in all cubes.

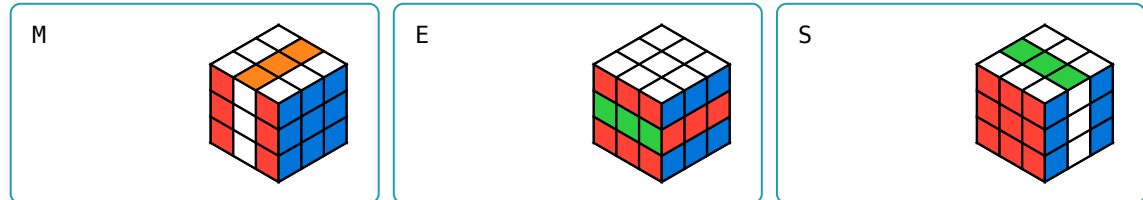
The maximum depth is the cube size minus 1. If you want to rotate the opposite face use the corresponding notation, i.e., in a 4x4x4 cube a 4F rotation is not allowed, the correct notation is B'.



III.2 Central moves

The central layers have a special notation:

- **M**: “middle”, between L and R following L.
- **E**: “equator”, between U and D following D.
- **S**: “standing”, between F and B following F.



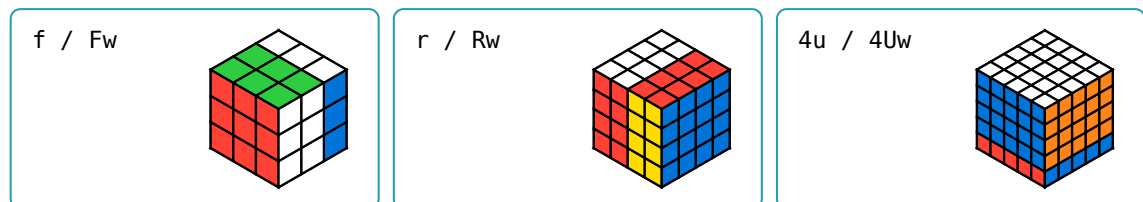
Central moves are only available on cubes with odd size.

III.3 Wide moves

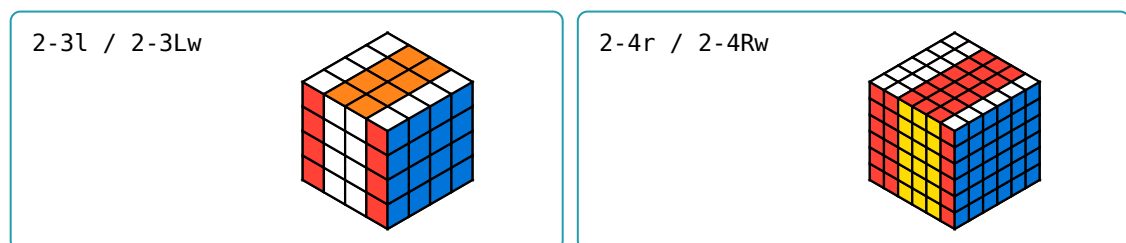
It is also possible to rotate more than one layer at the same time. This is especially useful for big cubes, but it is also used sometimes on a 3x3x3.

There are two equivalent notations: using lowercase letters or appending a **w** after the corresponding uppercase letter. By default, all layers between the outermost layer and the center (if the cube has odd size) are rotated, but this can be specified with a number before the move. For example, **3f** (or **3Fw**) rotates the first three front layers.

Just as before with the depth parameter, the maximum number of layers that can be moved with a wide move is equal to size - 1. A specific notations exists for rotating the entire cube that will be explained next.



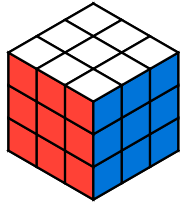
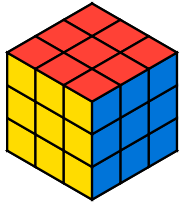
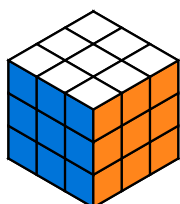
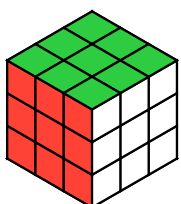
It is also possible to rotate multiple inner layers. To do so, write the first and last layers before the move separated with a dash.



III.4 Cube rotations

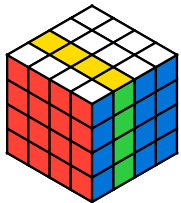
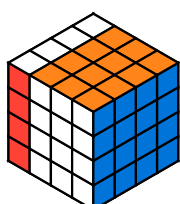
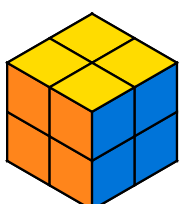
For a complete cube rotation the letters **x**, **y** and **z** are used. These movements do not alter the state of the cube, just the point of view.

- **x**: rotating the whole cube as if performing **R**.
- **y**: rotating the whole cube as if performing **U**.
- **z**: rotating the whole cube as if performing **F**.

Original state 	x 
y 	z 

III.5 Modifiers

Appending an apostrophe (') or a "2" to a move denotes an counterclockwise rotation or a double one (180°), respectively. These modifiers can be applied to any notation described above.

R' 	2F2 
3Rw' 	x2 

Part IV

Reference

IV.1 Custom types

IV.1.1 **cube**

cube is the main type in this package, as it represents the state of a cube. However, you should not create or modify instances manually, as functions such as `#draw_cube` require the cube to be in a consistent state. Instead, use `#cube` to create instances and `parse` to manipulate them.

```
(  
  {size} number  
  {f} array of color  
  {r} array of color  
  {u} array of color  
  {b} array of color  
  {l} array of color  
  {d} array of color  
)
```

The length of the dictionaries must be equal to the square of the size.

IV.1.2 **cube-colors**

This type is used when creating a cube to specify the color of each face.

```
(  
  {f} color  
  {r} color  
  {u} color  
  {b} color  
  {l} color  
  {d} color  
)
```

Not all keys need to be present for a valid **cube-colors** element.

IV.1.3 **cube-state**

This type is used when creating a cube to specify the pieces of each face.

```
(
  {f} array of color
  {r} array of color
  {u} array of color
  {b} array of color
  {l} array of color
  {d} array of color
)
```

Not all keys need to be present for a valid `cube-state` element.

IV.2 Functions

```
#apply          #draw_cube      #rotate_cube
#cube           #draw_flat      #rotate_layer
```

#apply(`{cube}`, `{alg}`, `{inverted}`): `false` → `cube`

Returns a new cube after applying the algorithm.

Argument — `{cube}` `cube`

The cube to apply the algorithm.

Argument — `{alg}` `str`

A string containing the algorithm to apply, following the notation specified in [Section III](#).

Argument — `{inverted}`: `false` `bool`

Whether it should be applied in inverse order.

This is useful when representing algorithms that, applied to the resulting cube, will solve it.

#cube(`{size}`: `3`, `{colors}`: `auto`, `{state}`: `auto`) → `cube`

This function creates a `cube` given the size, colors, and/or state. This is the correct way to create `cube` instances.

The `{state}` argument lets you specify all the pieces (colors) of a face. Note that if a face is specified with this argument, it must be fully specified, and the corresponding color given in `{colors}` is not used.

Argument — `{size}`: `3` `int`

The size of the cube. The default value is 3, i.e., a standard cube.

Argument

`{colors}: auto``cube-colors` | `auto`

Colors of each face of the cube.

It must be a `cube-colors`, although it does not need to contain all the keys. In that case, the default color value will be used.

The default values for each face are:

```
(
  f: red,
  r: blue,
  u: white,
  b: orange,
  l: green,
  d: yellow
)
```

Argument

`{state}: auto``cube-state` | `auto`

The state of the cube, the default is the solved state. See [Section II.1](#) for more details and examples.

```
#draw_cube(
  {cube},
  {x}: 35.264deg,
  {y}: 45deg,
  {z}: 0deg,
  {length}: 20pt
)
```

Draws a cube

Argument

`{cube}``cube`

the cube to draw.

Argument

`{x}: 35.264deg``angle`

the x angle.

Argument

`{y}: 45deg``angle`

the y angle.

Argument

`{z}: 0deg``angle`

the z angle.

#draw_flat(**{cube}**, (**length**): **20pt**)

Draws a flat cube.

Argument

{cube}

cube

The cube to draw.

#rotate_cube(**{cube}**, (**axis**))

Rotates a cube.

It returns the cube after applying the rotation. It is used for applying the rotations explained in [Section III.4](#).

Argument

{cube}

cube

The cube to apply the rotation.

Argument

{axis}

str

The axis that will follow the rotation. Must be one of ("x", "y", "z").

#rotate_layer(**{cube}**, (**index**), (**depth**): **1**, (**n**): **1**) → **cube**

Rotates a sinle layer of a cube.

It returns the cube after applying the rotation. It is used for applying the 1-layer rotations explained in [Section III.1](#).

Argument

{cube}

cube

The cube to apply the rotation.

Argument

{index}

str

The layer to rotate, must be one of ("f", "r", "u", "b", "l", "d").

Argument

{depth}: 1

int

The depth of the layer to rotate, by default 1, i.e., the outermost face.

It cannot be greater or equal the size of the cube.

Argument

{n}: 1

int

The number of rotations to apply, by default 1.

Part V

Index

A

`#apply` 5, 11

C

`#cube` 4, 10, 11

`cube` 10

`cube-colors` 10

`cube-state` 11

D

`#draw_cube` 10, 11, 12

`#draw_flat` 11, 13

R

`#rotate_cube` 6, 11, 13

`#rotate_layer` 6, 11, 13